Adaptive Learning System for CS Education ("Google Maps for Learning")

Research Report

Faiz Mustansar

January 5 – April 2, 2026

Brief Summary

This project investigates the design and implementation of an adaptive learning system for computer science (CS) education. The system estimates a learner's current knowledge state using Bayesian Knowledge Tracing, represents CS topics as a prerequisite graph, generates a personalized learning route toward a target goal, and dynamically updates that route as new evidence of learning is observed. The research component draws on peer-reviewed literature in adaptive learning, intelligent tutoring systems (ITS), student modeling, and knowledge tracing to motivate and justify implementation decisions related to knowledge representation, assessment design, routing logic, and evaluation. The project has produced a fully functional MVP: a dockerized web application implementing the complete adaptive learning loop, validated through unit and integration tests and end-to-end walkthroughs of the learner flow.

Contents

1 Background **3**
- 1.1 Adaptive Learning and Intelligent Tutoring Systems 3
- 1.2 Student Modeling and Knowledge Tracing 4
- 1.3 CS Knowledge Structure and Prerequisite Representation 6
- 1.4 Course Chatbots and General-Purpose Generative AI (In Progress) 8

2 Proposed System Design (System Design Only) **9**
- 2.1 Design Goals and Scope 9
- 2.2 System Architecture Overview 14
- 2.3 End-to-End Learning Flow 15
- 2.4 Core Data Model 17
- 2.5 Student Model Service: BKT Update 18
- 2.6 Routing Engine and Rerouting Policy 19
- 2.7 Learner User Interface (Conceptual) 20

2.8 Deployment and Infrastructure (Minimal) 21

3 Evaluation Plan **21**
3.1 BKT Sanity Checks 21
3.2 Routing and Decision Policy Validation 22
3.3 End-to-End Walkthrough 23
3.4 Limitations of This Evaluation 23

4 Risks and Timeline **23**
4.1 Risks and How They Played Out 23
4.2 Timeline 25

5 Conclusion **25**

1 Background

1.1 Adaptive Learning and Intelligent Tutoring Systems

Adaptive learning systems aim to personalize instruction by adapting content selection, sequencing, feedback, and pacing to individual learners. A central idea across the literature is that adaptation is driven by a model of the learner, which is updated using evidence from learner interactions and used to decide what the learner should do next [1, 2]. Unlike static curricula, adaptive systems continuously incorporate new evidence (e.g., correctness, attempt history, time-on-task) to provide recommendations that better reflect the learner's evolving knowledge state [1].

A major research lineage formalizing adaptive learning is Intelligent Tutoring Systems (ITS). ITS research typically decomposes instructional systems into four core components: (i) a *domain model* representing the target knowledge or skills, (ii) a *student model* representing what the learner currently knows, (iii) a *pedagogical model* specifying instructional decisions such as feedback or task selection, and (iv) an *interface model* that mediates interaction with the learner [2, 3]. This decomposition is particularly useful for implementation-focused projects, as it clarifies which parts of the system estimate learning and which parts act on those estimates.

VanLehn further distinguishes between two adaptation loops within tutoring systems: an *outer loop*, which selects the next task or learning activity, and an *inner loop*, which provides step-level guidance during problem solving [3]. For adaptive systems that emphasize learning path planning, such as the "Google Maps for learning" approach, the outer loop plays a central role by determining sequencing and rerouting decisions. Prior work suggests that robust outer-loop adaptation can yield meaningful benefits even when inner-loop guidance is relatively simple, making it a practical focus for term-scale prototypes [3, 2].

Adaptive learning systems rely heavily on student modeling techniques to infer latent knowledge states from observable behavior. Knowledge Tracing models, which treat mastery as an unobserved variable inferred from learner responses, have been widely used to support mastery learning and adaptive sequencing decisions [4]. More recent work extends these ideas to large-scale and data-rich environments, demonstrating that continuous learner modeling can support both prediction and personalization [5]. These results motivate the integration of explicit student models into adaptive CS learning systems.

From an implementation perspective, adaptive systems must balance theoretical expressiveness with practical constraints such as data quality, interpretability, and robustness. Large-scale tutoring platforms highlight the importance of transparent decision policies and defensible inference mechanisms, particularly in educational contexts where instructors and learners must trust system behavior [6]. These considerations align with the goals of an implementation-focused capstone project that prioritizes interpretable models, explicit knowledge structures, and verifiable adaptation logic.

1.2 Student Modeling and Knowledge Tracing

Adaptive learning systems require a representation of what a learner knows in order to make defensible sequencing and feedback decisions. This representation is commonly called a *student model*: a computational estimate of the learner's knowledge state that is updated from observable interaction evidence (e.g., responses to questions, attempts on exercises, or programming submissions). A key challenge is that knowledge is *latent* (unobserved); systems therefore infer mastery probabilistically from performance traces rather than directly measuring understanding [4]. In knowledge tracing, the objective is typically to estimate a learner's evolving mastery over a set of skills/knowledge components (KCs) and use these estimates to guide instruction and sequencing [4].

Bayesian Knowledge Tracing (BKT). Bayesian Knowledge Tracing (BKT) is a widely used method for modeling student knowledge in intelligent tutoring and adaptive learning systems. In its basic form, BKT treats each Knowledge Component (KC) as a hidden (unobserved) state that represents whether a student has mastered a specific skill or not. At any point in time, the system does not know this state directly and must estimate it based on the student's responses [4].

BKT is commonly modeled as a two-state process where a student moves from *not mastered* to *mastered*. After each attempt at a question or exercise, the system updates its belief $P(L_t)$, which represents the probability that the student has mastered the skill at time t. This approach is especially suitable for implementation-focused systems because it is simple, interpretable, and supports clear decision rules, such as advancing the student when mastery is high or providing extra practice when mastery remains low [4].

In the standard BKT model, each skill is described using four probabilities:

- $P(L_0)$: the initial probability that the student already knows the skill
- $P(T)$: the probability that the student learns the skill after an attempt
- $P(G)$: the probability that the student answers correctly by guessing
- $P(S)$: the probability that the student answers incorrectly despite knowing the skill

When a student responds to an item at time t, where $Correct_t \in \{0, 1\}$ indicates whether the response was correct, BKT first updates the probability of mastery using Bayes' rule:

$$P(L_t \mid Correct_t) = \begin{cases} \frac{P(L_{t-1})(1-P(S))}{P(L_{t-1})(1-P(S))+(1-P(L_{t-1}))P(G)} & \text{if } Correct_t = 1 \\ \frac{P(L_{t-1})P(S)}{P(L_{t-1})P(S)+(1-P(L_{t-1}))(1-P(G))} & \text{if } Correct_t = 0 \end{cases}$$

After this update, the model accounts for the possibility that the student learned the skill as a result of the attempt:

$$P(L_t) = P(L_t \mid Correct_t) + \big(1 - P(L_t \mid Correct_t)\big)P(T)$$

In simple terms, this process uses each student response to update how confident the system is that the student understands the skill, while accounting for guessing, mistakes, and learning over time [4].

BKT in CS and Programming Contexts. BKT was originally studied in tutoring contexts where learners practiced procedural skills over time, including programming-oriented tutoring environments [4]. In CS education, BKT is particularly relevant because systems can instrument repeated practice opportunities and map them to KCs (e.g., conditionals, loops, indexing, recursion). Evidence may come from structured questions or from programming exercises (test-case correctness, compilation success, repeated attempts). Modern programming learning platforms also produce richer traces than binary correctness, motivating extensions that incorporate additional signals or alternative models when feasible [5]. For a capstone-scale implementation, BKT remains a strong baseline because it provides a clear, testable bridge between observed performance and actionable sequencing logic [4].

Implementation Guidance (Prototype-Oriented). For an implementation-focused prototype, BKT can be implemented reliably by specifying a clean data schema and an explicit KC mapping:

- **Define KCs:** Decide the set of skills (KCs) the system will track (e.g., 10–30 KCs for an intro CS unit). Each KC gets its own BKT state and parameters [4].
- **Map items to KCs:** Each quiz item or programming exercise should be tagged with one (or a small set of) KCs. The simplest approach is an expert-defined mapping for the initial prototype.
- **Collect evidence:** Store per-attempt logs: (learner_id, item_id, kc_id, timestamp, correctness, optional features such as attempts/time/compilation status).
- **Initialize parameters:** Start with plausible priors (e.g., moderate guess/slip; conservative learning rate) and refine with small-scale fitting if data permits. If fitting is not feasible within the term, parameters should be justified via literature and validated through sanity checks (Section **??**) [8].
- **Decision thresholds:** Implement a transparent policy such as: advance if $P(L_t) \geq \tau_{mastery}$, remediate if $P(L_t) \leq \tau_{low}$ after k opportunities.

Figure 1 illustrates the closed-loop logic expected in an adaptive system: evidence updates a student model, which drives a decision policy for next actions. Figure **??** emphasizes BKT's key assumption: mastery is latent, so responses are treated as noisy observations (guess/slip) [4].

Limitations and Practical Pitfalls. Despite its usefulness, BKT has several limitations that matter in practice. First, standard BKT assumes binary mastery and typically binary observations per KC opportunity, which can be reductive for CS tasks where partial understanding and multi-step reasoning are common. Second, parameters can be sensitive and may yield unrealistic mastery curves if poorly chosen or fit; analytic work shows that the behavior of BKT can collapse into limited functional forms, and different parameter sets may produce similar predictions, raising identifiability concerns [8]. Third, the independence assumptions in classic BKT (e.g., skills modeled separately, opportunities treated similarly) can be violated in CS domains where skills interact (e.g., loops and arrays) and where item difficulty varies significantly.

A common direction in the literature is to relax some of these assumptions by individualizing parameters across students or incorporating richer features. For example, individualized BKT variants introduce student-specific parameters to better capture heterogeneity in prior knowledge or learning rates, improving predictive performance compared to skill-only parameterization [9]. More expressive approaches such as Deep Knowledge Tracing replace explicit BKT structure with sequence models that can capture complex dependencies; however, these methods often trade interpretability for accuracy and require more data and careful validation [5]. For this project, BKT is a defensible baseline because it is transparent and implementable, while still allowing incremental extensions if sufficient data and time become available [4, 8, 9].

1.3 CS Knowledge Structure and Prerequisite Representation

Computer science (CS) domains are naturally structured around prerequisite relationships: learners must understand simpler concepts before progressing to more complex abstractions (e.g., variables before conditionals, conditionals before loops, loops before recursion). This hierarchical dependency structure makes CS education particularly well-suited to graph-based knowledge representations and path-planning approaches [11, 10].

From an adaptive learning perspective, the ability to explicitly encode prerequisites enables systems to reason about *where a learner is*, what they are *ready to learn next*, and how to *reroute* instruction when learning difficulties are detected. This aligns directly with the "Google Maps for learning" metaphor: the learner's current mastery state determines their location on a map, while prerequisite constraints define which routes are legally traversable [1].

Knowledge Components (KCs). A foundational design decision is how to decompose CS content into *knowledge components* (KCs). KCs are the smallest units of knowledge or skill that can be meaningfully practiced and assessed independently [11]. In introductory CS contexts, examples include:

- Variables and assignment

- Boolean expressions
- Conditional branching
- Iteration (for/while loops)
- Functions and parameter passing
- Arrays or lists
- Recursion

Each KC serves as a node in the system's domain model and is tracked independently by the student model (e.g., one BKT instance per KC). The granularity of KCs represents a trade-off: coarse KCs reduce modeling complexity but may obscure specific misconceptions, while overly fine-grained KCs increase annotation and parameterization burden [11, 10]. For a term-scale prototype, an expert-defined KC set grounded in standard CS curricula provides a defensible and implementable compromise.

Prerequisite Graph Representation. Once KCs are defined, prerequisite relationships can be encoded as a directed graph $G = (V, E)$, where each node $v \in V$ corresponds to a KC and each directed edge $(u \to v) \in E$ indicates that mastery of u is required before learning v. In many introductory CS domains, this graph can be approximated as a directed acyclic graph (DAG), simplifying routing and validation logic [11].

Figure 3 illustrates a simplified prerequisite graph for introductory programming concepts. Such a structure allows the system to: (i) determine which nodes are currently available given a learner's mastery state, (ii) identify unmet prerequisites for a target goal, and (iii) generate valid learning routes that respect dependency constraints.

Learning Path Planning as Graph Routing. Given a prerequisite graph and a learner's estimated mastery state, learning path generation can be formalized as a constrained routing problem. The system seeks a path from the learner's current location (set of mastered nodes) to a target goal node, subject to prerequisite constraints. In its simplest form, this can be implemented as rule-based prerequisite satisfaction: repeatedly select the nearest unmet prerequisite until all prerequisites for the target are satisfied [1].

More advanced formulations treat routing as a graph search problem, where nodes have associated costs derived from mastery probabilities, topic difficulty, or estimated learning effort. Such formulations allow the system to compare multiple valid routes and select one that optimizes a chosen objective (e.g., minimal expected remediation) [1]. For a capstone-scale system, a rule-based or breadth-first prerequisite expansion provides sufficient adaptivity while remaining transparent and debuggable.

Integration with Student Modeling. Prerequisite graphs interact tightly with student models such as BKT. Mastery estimates determine which nodes are considered "complete," "available," or "blocked." When a learner repeatedly fails to demonstrate mastery on a KC, the system can reroute by moving backward in the graph to reinforce prerequisites. Conversely, when mastery probabilities exceed a threshold, the system may unlock downstream nodes and advance the learning route. This explicit coupling between graph structure and mastery inference operationalizes adaptive sequencing in a way that is both explainable and implementable [4, 11].

Limitations and Design Risks. Despite their usefulness, prerequisite graphs impose simplifying assumptions. First, real learning dependencies may not be strictly hierarchical; some CS concepts are mutually reinforcing rather than linearly ordered. Second, expert- defined graphs risk encoding instructor bias and may not reflect how learners actually acquire skills [11]. Third, prerequisite satisfaction alone does not capture differences in instructional effectiveness or learner preferences.

To mitigate these limitations, this project treats the prerequisite graph as an initial scaffold rather than a ground truth. The graph is intended to be small, transparent, and revisable, allowing future extensions such as data-driven edge validation or dynamic prerequisite relaxation. Within the scope of this project, a carefully designed expert graph provides a practical foundation for adaptive routing while keeping the system interpretable and robust.

1.4 Course Chatbots and General-Purpose Generative AI (In Progress)

In recent years, several universities and educational platforms have introduced course-specific chatbots to support student learning. These systems are typically designed to answer frequently asked questions, summarize course materials, or provide clarifications about assignments and deadlines. In some cases, they are trained on lecture slides, textbooks, or discussion forum content to offer context-aware responses. While useful as instructional support tools, these course chatbots generally do not model student knowledge or provide structured, adaptive tutoring.

Unlike Intelligent Tutoring Systems, most university-deployed chatbots do not maintain a persistent student model, track mastery over individual skills, or adapt content sequencing based on learner performance. Their primary function is information retrieval or explanation rather than instructional decision-making. As a result, these systems offer limited guidance on what a student should learn next or how their understanding evolves over time.

General-purpose generative AI systems such as ChatGPT, Claude, and Gemini further extend conversational capabilities by providing natural language explanations, code examples, and conceptual guidance across a wide range of topics. These tools have demonstrated strong performance in explaining program-

ming concepts and assisting with debugging. However, they operate without explicit knowledge of a learner's prior understanding, progress, or misconceptions. Responses are generated independently for each interaction, with no guaranteed consistency or alignment to a structured curriculum.

From an educational modeling perspective, general-purpose language models lack explicit representations of prerequisite relationships, mastery thresholds, and learning objectives. While they may simulate tutoring behavior through dialogue, they do not implement formal student modeling or adaptive sequencing policies. This limits their ability to provide reliable learning paths or to justify instructional decisions in a transparent way.

The proposed system differs from both course chatbots and general-purpose generative AI by combining adaptive student modeling with an explicit knowledge map of CS concepts. Rather than answering isolated questions, the system tracks mastery at the level of individual knowledge components and uses this information to generate and update personalized learning routes. Generative AI tools may be used as supportive components (e.g., explanation or feedback generation), but the core adaptivity logic is driven by interpretable models such as Bayesian Knowledge Tracing and prerequisite-based routing.

This distinction positions the system closer to classical intelligent tutoring systems while incorporating modern interaction paradigms. By separating instructional decision-making from explanation generation, the system aims to provide adaptive guidance that is both personalized and pedagogically grounded, addressing limitations observed in existing chatbot-based educational tools.

2 Proposed System Design (System Design Only)

2.1 Design Goals and Scope

This section specifies a term-scale, implementation-ready system design for an adaptive CS learning platform. The design prioritizes (i) interpretability, (ii) explicit knowledge representation via a prerequisite graph, and (iii) transparent decision-making for sequencing and rerouting. Adaptation is driven by a student model (Bayesian Knowledge Tracing) operating over a set of instructor-defined Knowledge Components (KCs), consistent with established ITS decompositions and outer-loop sequencing logic [2, 3, 4].

Prototype scope (v1). The v1 system supports goal selection, diagnostic initialization, quiz/exercise attempts, mastery updates, and prerequisite-respecting route generation. To reduce ambiguity in mastery updates and ensure clean instrumentation, each activity item is mapped to exactly one KC in v1.

Out of scope (v1). Deep sequence models (e.g., DKT), automated KC discovery, multi-KC credit assignment, and sophisticated step-level tutoring (inner-

loop guidance) are deferred in favor of reliable outer-loop routing and mastery tracking [5, 3].

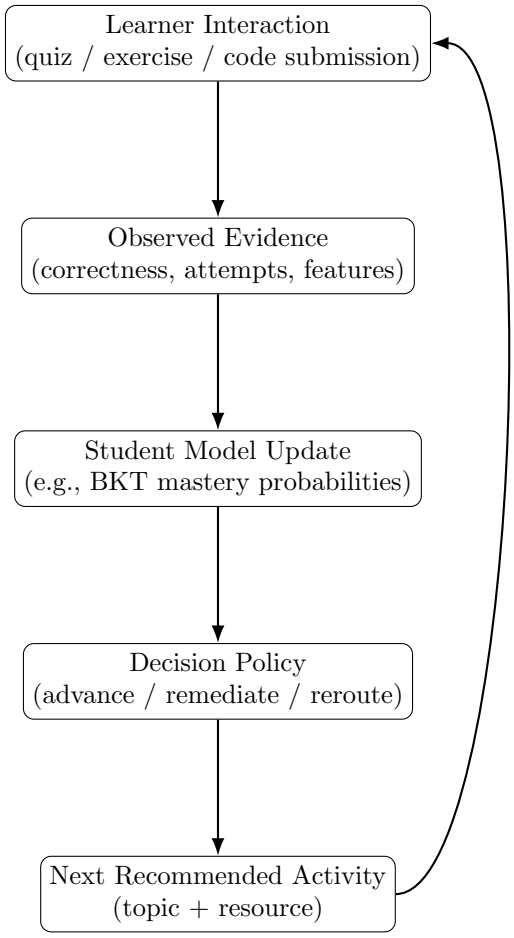

Figure 1: Closed-loop adaptation: interaction evidence updates the student model, which drives next-step decisions.

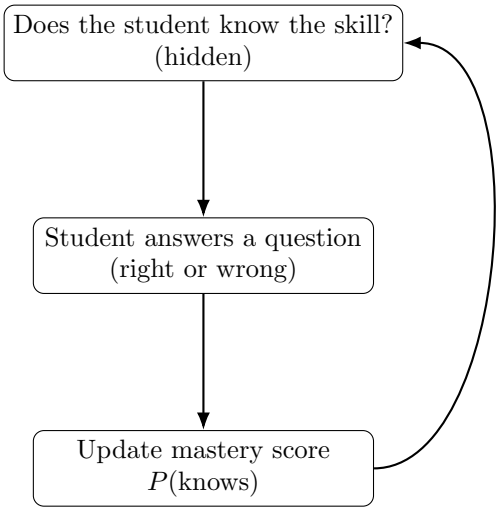

Figure 2: Simple view of BKT: the system cannot directly see knowledge, so it uses answers to update a mastery score over time.

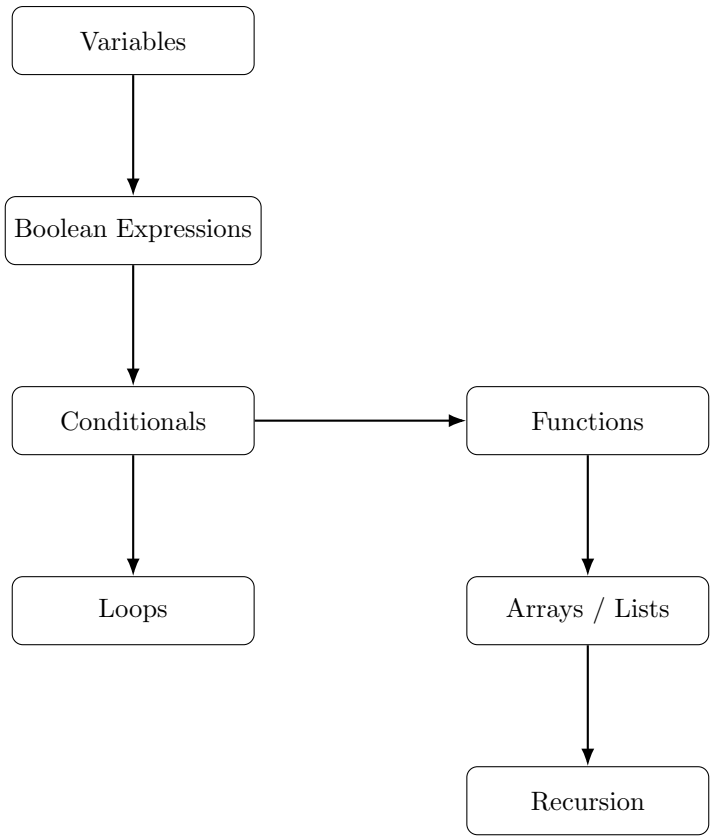

Figure 3: Example prerequisite graph for introductory CS concepts. Nodes represent knowledge components; edges represent prerequisite relationships.

2.2 System Architecture Overview

Figure 4 presents the system-level architecture. Learners interact with a map-based UI, while instructors/admins define KCs, prerequisite edges, and an item bank. A backend API mediates all access, persists events, and orchestrates routing and student-model updates.

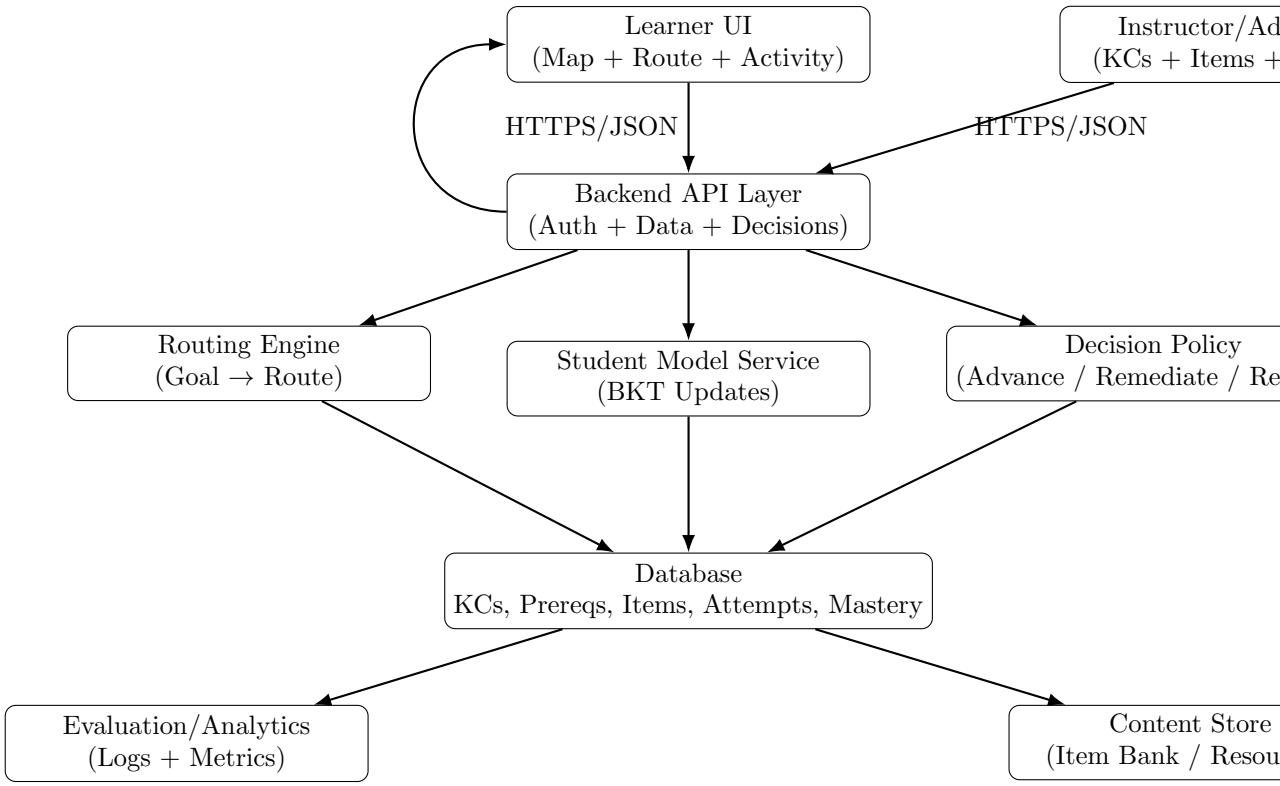

Figure 4: System-level architecture for the adaptive learning platform.

Key services.

- **Routing Engine:** constructs prerequisite-respecting learning routes from current mastery to a goal KC.
- **Student Model Service:** maintains per-learner mastery probabilities using BKT updates [4].
- **Decision Policy:** converts mastery signals into actions (advance, remediate, reroute) using transparent thresholds.

2.3 End-to-End Learning Flow

Figure 5 describes the outer-loop learning flow. The learner selects a goal KC, completes a diagnostic to initialize mastery priors, follows the recommended route, and repeatedly updates mastery after each attempt. The system reroutes when evidence indicates persistent low mastery.

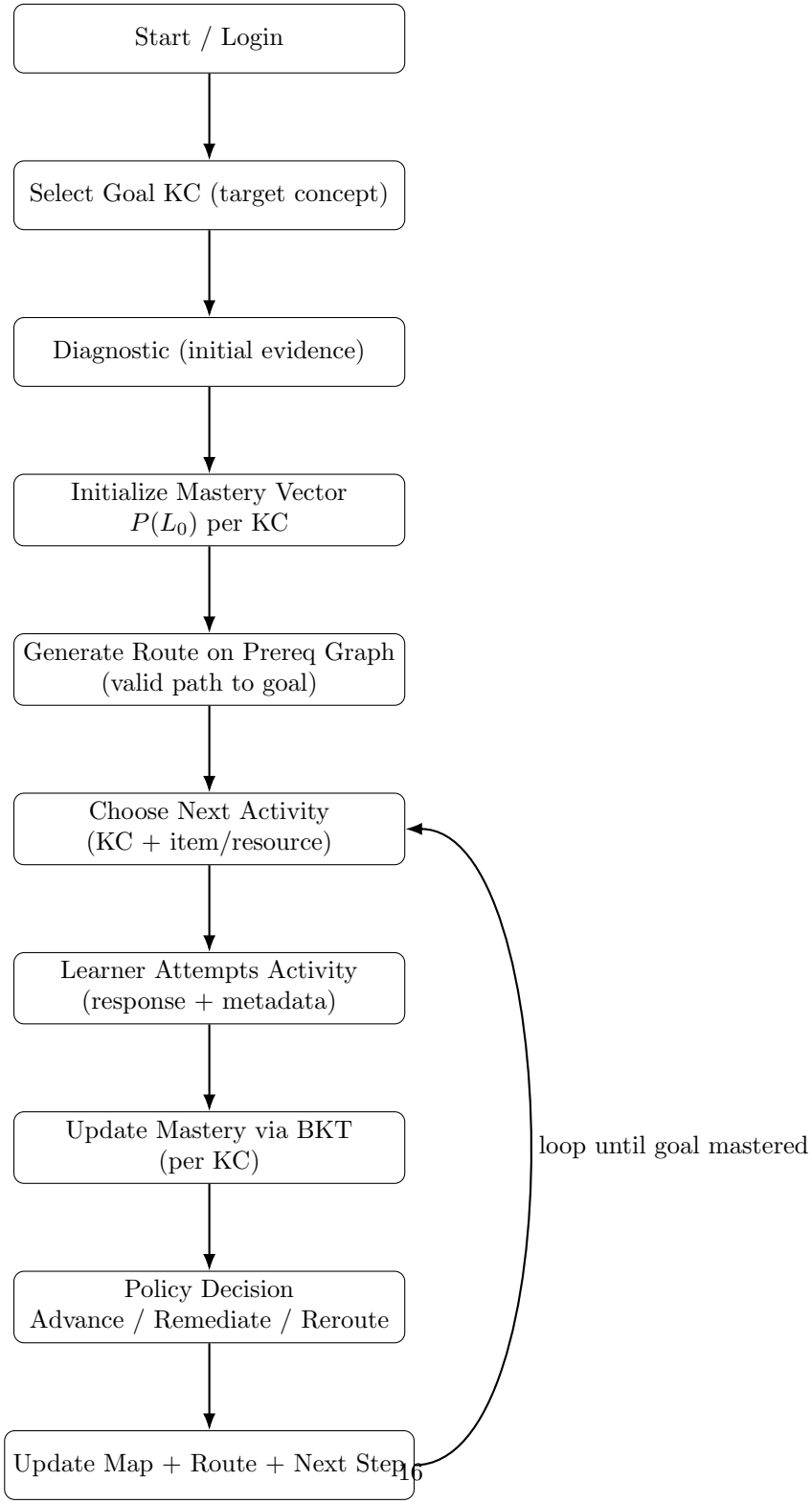

Figure 5: End-to-end learning flow: diagnostic → route → practice → BKT update → decision → reroute.

Diagnostic initialization. The diagnostic provides initial evidence to set $P(L_0)$ values per KC. In v1, the diagnostic can be implemented as a small set of KC-tagged items; mastery priors are initialized using either fixed priors or diagnostic-derived posteriors (same BKT update rule).

2.4 Core Data Model

The platform uses a minimal relational schema that supports trace logging (attempts), mastery estimation, and route computation. Figure 6 summarizes the core entities.

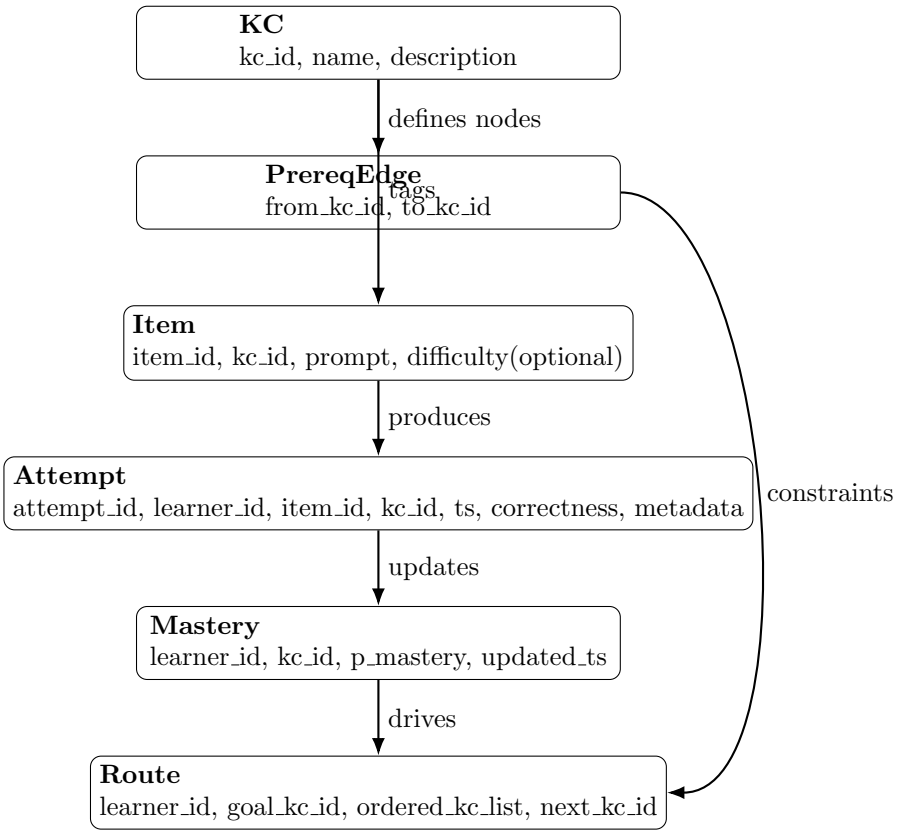

Figure 6: Core data objects used by the system.

Item-to-KC mapping (v1 constraint). Each item is tagged to exactly one KC. This ensures that each attempt triggers a single BKT update and avoids multi-skill credit assignment in the prototype.

2.5 Student Model Service: BKT Update

Mastery is modeled independently per KC using Bayesian Knowledge Tracing [4]. Each attempt produces a binary observation (correct/incorrect) and updates $P(L_t)$ for the attempted KC using Bayes' rule with guess and slip, followed by the learning transition. Figure 7 shows the update micro-flow.

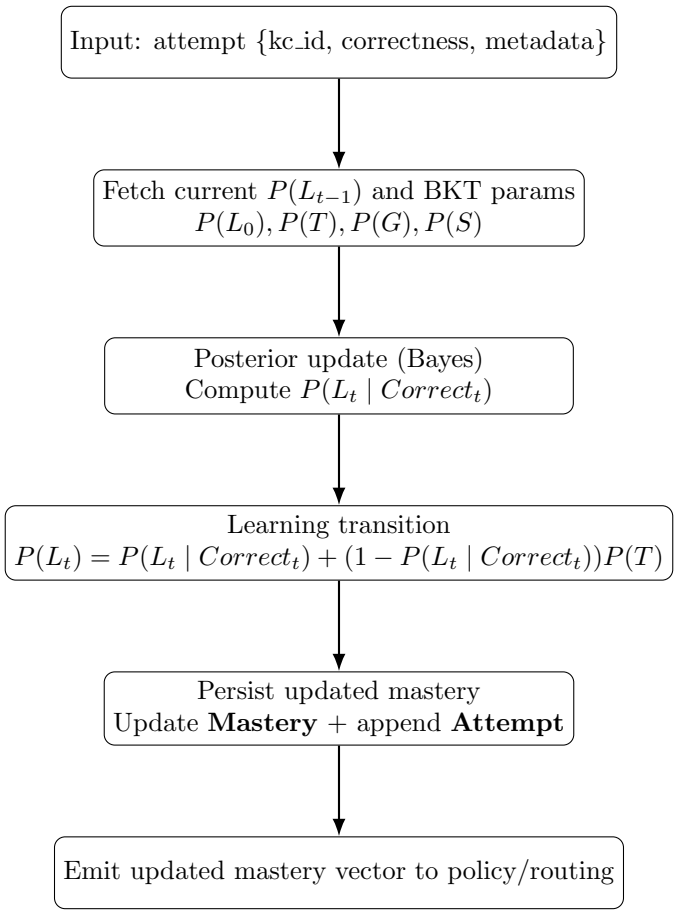

Figure 7: Micro-flow for updating mastery probabilities using BKT after each learner attempt.

BKT parameterization. In v1, BKT parameters may be set globally (shared across KCs) or per-KC using instructor-configured defaults. Parameter fitting is optional; if fitting is not feasible within the term, parameters are justified via literature and validated via sanity checks on mastery trajectories [4, 8].

2.6 Routing Engine and Rerouting Policy

Routing generates an ordered list of KCs leading from the learner's current mastery state to a goal KC while respecting prerequisite constraints. The system treats a KC as "mastered" when $P(L) \geq \tau_{mastery}$. Figure 8 summarizes the routing logic.

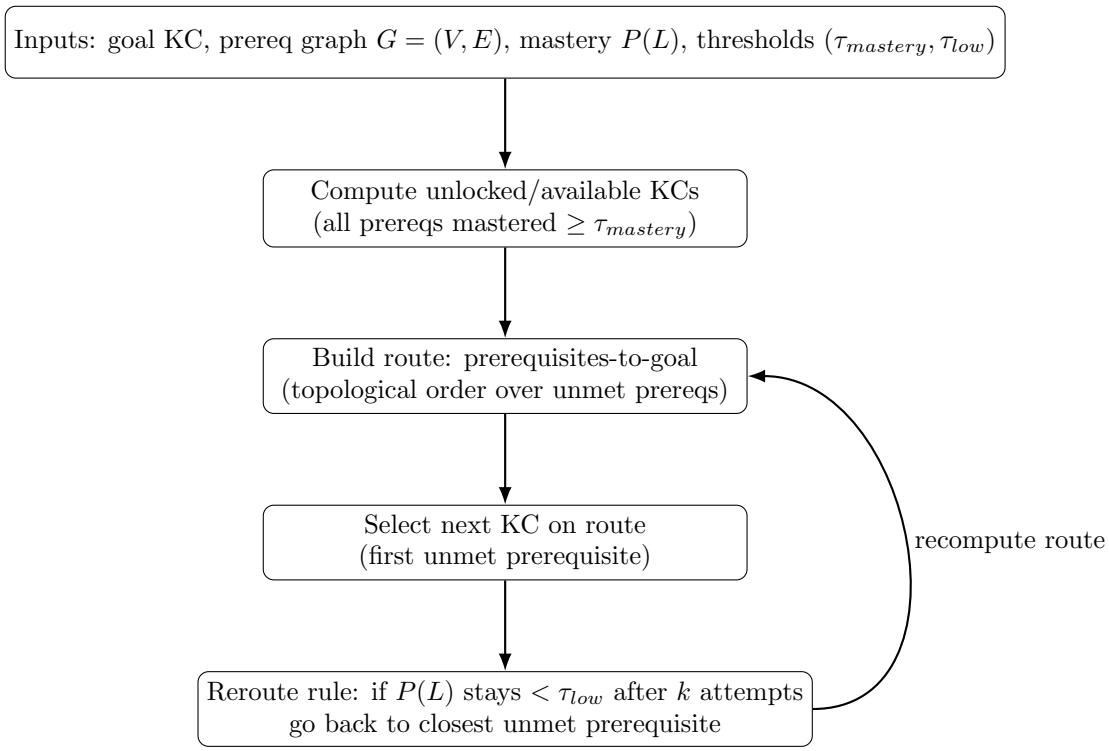

Figure 8: Routing and rerouting: prerequisite satisfaction and mastery-driven backtracking.

Route construction (v1). Given a goal KC, the engine computes the set of unmet prerequisites (backward graph traversal), topologically orders them, and recommends the first unmet prerequisite as the next KC. This yields a transparent, debuggable route consistent with prerequisite-based sequencing [1].

Rerouting rule (v1). If a learner repeatedly fails to improve mastery on the current KC (e.g., $P(L) < \tau_{low}$ after k attempts), the system selects the closest unmet prerequisite (or the nearest ancestor KC not mastered) and recomputes the route.

2.7 Learner User Interface (Conceptual)

The learner UI is designed to make adaptation understandable. Rather than recommending items in an opaque way, the UI visualizes the prerequisite map, indicates which nodes are unlocked, and explains why a KC is selected next. Figure 9 summarizes the primary screens.

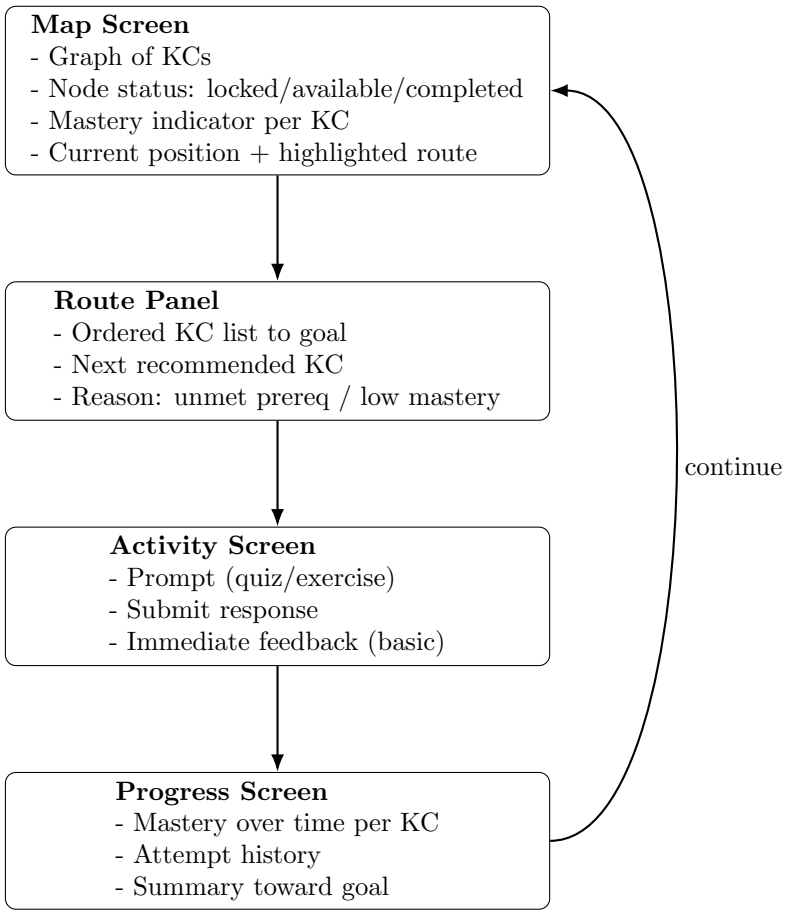

Figure 9: Conceptual UI flow for a learner: map → route → activity → progress.

Instructor/Admin UI (v1). The admin UI supports creating KCs, defining prerequisite edges, and authoring KC-tagged items. This enables rapid iteration on the domain model while keeping the prerequisite structure explicit and editable.

2.8 Deployment and Infrastructure (Minimal)

The system can be deployed using a standard web stack with a hosted frontend, an API service, and a managed relational database. Figure 10 illustrates a minimal deployment appropriate for a term-scale prototype.

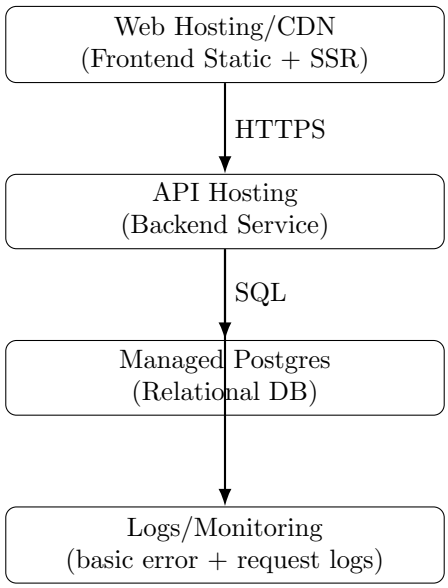

Figure 10: Minimal deployment setup suitable for a term-scale prototype.

Logging. All learner interactions are persisted as append-only attempt events to support debugging, reproducibility, and evaluation. Basic request/error logs are collected at the API layer.

3 Evaluation Plan

When I started thinking about evaluation, I wanted to be honest with myself about what I could actually measure within a single-term project without real learner populations. Rather than simulate an experiment I couldn't run, I focused on three concrete validation goals: (1) verifying that the BKT implementation behaves correctly, (2) confirming that the routing and decision logic matches the intended policy, and (3) demonstrating that the end-to-end system works as designed through structured manual walkthroughs.

3.1 BKT Sanity Checks

The first thing I validated was the BKT update function itself. I wrote unit tests that trace mastery trajectories under controlled conditions:

- A learner who answers every question correctly should cross $\tau_{mastery} = 0.95$ within a reasonable number of attempts given the default parameters ($P(L_0) = 0.10$, $P(T) = 0.10$, $P(G) = 0.25$, $P(S) = 0.10$). In practice, this happens in roughly 8–12 correct answers, which felt reasonable.
- A learner who answers every question incorrectly should see mastery stay low and eventually trigger the reroute condition ($P(L) < \tau_{low} = 0.40$ after $k = 3$ attempts).
- Mixed sequences (correct then incorrect) should produce intermediate trajectories that don't swing wildly, which they did.

These checks gave me confidence that the Bayesian update and learning transition were implemented correctly per the Corbett & Anderson formulation [4]. The parameters were taken directly from literature defaults rather than fit to data, which is a known limitation, but for a prototype without learner data, this is a reasonable starting point [8].

3.2 Routing and Decision Policy Validation

The routing engine was validated through integration tests covering the following cases:

- **Clean path to goal:** Given a learner with no mastered KCs targeting Recursion, the engine should return the full topologically-ordered path (Variables → Boolean Expressions → Conditionals → Loops → Functions → Recursion). It does.
- **Partial mastery:** If Variables and Boolean Expressions are already mastered, the route should begin at Conditionals. It does.
- **Reroute trigger:** After $k = 3$ attempts on a KC with $P(L) < 0.40$, the system should fall back to the nearest unmet prerequisite and recompute. This was verified both in unit tests and manually in the UI.
- **Goal already mastered:** If the learner already exceeds $\tau_{mastery}$ on the goal KC, no route should be generated. This edge case was handled correctly.

Each attempt response from the API also includes a plain-language `decision` field explaining *why* the system made the choice it did (e.g., "Your mastery of Conditionals is now 0.97, advancing to Loops"). I treated this as a lightweight explainability check: if the rationale read correctly and consistently, the underlying logic was likely sound.

3.3 End-to-End Walkthrough

I ran several manual walkthroughs of the full learner flow to verify the system behaves coherently in practice. The scenario I used most was:

1. Enter as a new learner with no prior history.
2. Select Recursion as the goal.
3. Work through the generated route, deliberately answering some questions incorrectly to trigger remediation and reroute.
4. Verify that the map updates in real time, the mastery bar moves correctly, and the next KC recommendation matches what the decision policy should output.

The system behaved as expected across all walkthroughs. The interactive knowledge graph (built with React Flow) rendered node states correctly: locked, in progress, and mastered nodes were visually distinct, and the AI tutor responses were contextually appropriate without ever interfering with the routing logic.

3.4 Limitations of This Evaluation

I want to be clear about what this evaluation does and does not show. Because I didn't have real learners, I can only verify correctness of implementation, not effectiveness as a learning tool. The BKT parameters were not fit to data, so mastery trajectories reflect literature defaults rather than calibrated estimates for this specific KC set. Whether $\tau_{mastery} = 0.95$ is the right threshold for advancing a learner in practice is an empirical question I wasn't able to answer in this term.

The most meaningful next step for evaluation would be a small pilot study with actual students, tracking whether the adaptive routes improve mastery acquisition relative to a fixed sequential curriculum. This would provide the kind of evidence that would let me iterate on thresholds, parameters, and routing heuristics with real signal rather than intuition.

4 Risks and Timeline

4.1 Risks and How They Played Out

At the start of the project I identified several risks that I thought were most likely to derail the work. Looking back, some of them materialized in roughly the way I expected, and a few turned out to be non-issues.

BKT identifiability and parameter sensitivity. This was the risk I was most concerned about going in. The literature is clear that BKT can produce similar mastery curves from very different parameter sets, and without real learner data to fit against, I had no way to know whether my chosen parameters were realistic [8]. In practice, I mitigated this by choosing conservative defaults from Corbett & Anderson [4] and manually checking that mastery trajectories "felt right" under a variety of simulated answer sequences. It worked well enough for a prototype, but I'm aware that this is the weakest part of the system from a scientific standpoint. Any future work would need real data and proper parameter fitting.

Graph routing correctness with multi-path prerequisites. The prerequisite graph has one branching case: both Loops and Arrays/Lists are prerequisites for Recursion via different paths (Conditionals → Loops → Functions → Recursion and Conditionals → Functions → Arrays/Lists → Recursion). I was worried that the backward BFS + topological sort approach would produce incorrect or ambiguous routes at the merge point. In the end, the implementation handled this cleanly; the topological ordering correctly serialized both branches, but it took a few iterations to get right.

Frontend complexity and state synchronization. I underestimated how much state management complexity would emerge on the frontend once routing decisions needed to update the map, the mastery bar, and the next-activity recommendation simultaneously. Using Zustand with localStorage persistence helped, but there were some tricky bugs where stale state from a previous session would interfere with a new route. I fixed these, but it cost more time than I had budgeted.

AI tutor integration interfering with routing. Early on I was worried that integrating an LLM tutor might complicate the routing logic or create confusing interactions where the tutor's advice contradicted the system's recommendation. I resolved this cleanly by treating the tutor as a completely separate service: it reads the current KC context but has no write access to mastery, routes, or attempts. The routing logic is entirely deterministic and unaffected by whatever the tutor says. This architectural decision turned out to be the right one.

Risks that did not materialize. Database schema migrations were surprisingly smooth; Alembic handled all incremental changes without requiring manual intervention. Docker Compose setup, which I anticipated would be fiddly, worked reliably across clean installs. And the React Flow graph library handled the prerequisite DAG rendering much more gracefully than I expected.

4.2 Timeline

The project ran from early January to early April 2026 over approximately 13 weeks.

- **Weeks 1–2 (Jan 5–18):** Literature review. Read Corbett & Anderson (BKT), VanLehn (outer/inner loop), Brusilovsky (adaptive hypermedia), and surveyed DKT. Drafted the background sections of this report.
- **Weeks 3–4 (Jan 19 – Feb 1):** System design. Defined the KC set, prerequisite graph structure, data model, and API contracts. Wrote up the proposed system design section.
- **Weeks 5–6 (Feb 2–15):** Core backend. Implemented the FastAPI skeleton, SQLAlchemy models, Alembic migrations, and the BKT service. Wrote unit tests for the update function.
- **Weeks 7–8 (Feb 16 – Mar 1):** Routing engine and decision policy. Implemented backward BFS route generation, the advance/remediate/reroute policy, and the seed data (7 KCs, 7 edges, 28+ items).
- **Weeks 9–10 (Mar 2–15):** Frontend. Built the Next.js app with React Flow knowledge graph, Zustand store, and the core learner flow (landing → goal → map → practice → progress).
- **Weeks 11–12 (Mar 16–29):** Integration, AI tutor, admin UI, Docker Compose setup. Wired frontend to backend, added the optional LLM tutor panel, built the admin interface for managing KCs and items.
- **Week 13 (Mar 30 – Apr 2):** Testing, evaluation walkthroughs, documentation, and report completion.

5 Conclusion

When I started this project I had a fairly abstract idea: build something like "Google Maps for learning", a system that knows where a learner is, knows where they want to go, and generates a route that adjusts as they move through the material. Thirteen weeks later, that idea is a working system.

The MVP implements the full adaptive loop as designed. A learner picks a goal KC, the system builds a personalized prerequisite-respecting route using backward BFS and topological sort, quiz items are served one at a time, and mastery is updated after every attempt using Bayesian Knowledge Tracing. The decision policy (advance when $P(L) \geq 0.95$, reroute when $P(L) < 0.40$ after three attempts) runs cleanly and produces routing decisions that are both sensible and explained to the learner in plain language. The interactive knowledge graph updates in real time, and the optional AI tutor (Claude by default) provides contextual explanation without ever touching the routing logic.

What I'm most satisfied with is that the system is genuinely interpretable. Every decision the system makes, whether to advance, remediate, or reroute, comes with a plain-language rationale that a learner can read and actually understand. This was a deliberate design priority from the beginning, and it held up through implementation. I didn't have to sacrifice transparency to make the adaptation work.

That said, I want to be honest about the limitations. The BKT parameters are literature defaults, not fitted values. Without real learner data, I have no way to know whether $\tau_{mastery} = 0.95$ is the right bar, or whether $P(T) = 0.10$ reflects how quickly students actually learn the KCs in the seed data. The expert-defined prerequisite graph is reasonable, but it's still one person's model of how CS concepts depend on each other, and a data-driven edge validation would strengthen it considerably. And the evaluation, while rigorous in terms of correctness testing, cannot answer the question that matters most: does this system actually help students learn better than a fixed curriculum would?

These are exactly the right questions to ask next. A small pilot study with real introductory CS students, tracking mastery trajectories, comparing adaptive routes against sequential ones, and gathering qualitative feedback on explainability, would provide the signal needed to move from a working prototype to a validated system. The infrastructure is built and ready for that kind of study.

Looking at the research literature I surveyed at the start, I think the design choices held up well. BKT turned out to be exactly the right model for a first implementation: transparent, implementable, and expressive enough to drive meaningful routing decisions without requiring data I didn't have. VanLehn's outer-loop framing was the right lens for thinking about the system's core responsibility [3]. And Brusilovsky's observation that explicit prerequisite graphs enable both rerouting and explainability [1] proved true in practice; the graph structure is the thing that makes the system's recommendations legible to learners, not just technically correct.

The project met the goals I set out with. The adaptive learning system I described in January exists as a functional, tested, dockerized application in April. I learned a lot building it, from knowledge tracing and graph-based routing to how hard it is to synchronize frontend state with real-time backend decisions, and I think it's a solid foundation for future work in adaptive CS education.

References

[1] P. Brusilovsky, *Adaptive Hypermedia*, User Modeling and User-Adapted Interaction, vol. 11, no. 1–2, pp. 87–110, 2001. Available: https://link.springer.com/article/10.1023/A:1011143116306.

[2] B. P. Woolf, *Building Intelligent Interactive Tutors: Student-Centered Strategies for Revolutionizing E-Learning*, Morgan Kaufmann, San Francisco, CA, 2009.

[3] K. VanLehn, *The Behavior of Tutoring Systems*, International Journal of Artificial Intelligence in Education, vol. 16, no. 3, pp. 227–265, 2006. Available: https://www.cs.colorado.edu/~mozer/Teaching/syllabi/6872/readings/VanLehn06.pdf.

[4] A. T. Corbett and J. R. Anderson, *Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge*, User Modeling and User-Adapted Interaction, vol. 4, no. 4, pp. 253–278, 1995. Available: https://link.springer.com/article/10.1007/BF01099821.

[5] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, *Deep Knowledge Tracing*, Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015. Available: https://stanford.edu/~cpiech/bio/papers/deepKnowledgeTracing.pdf.

[6] M. Feng, N. Heffernan, and K. Koedinger, *Addressing the Assessment Challenge with an Online System that Tutors as it Assesses*, Proceedings of the 7th International Conference on Educational Data Mining (EDM), 2014. Available: https://educationaldatamining.org/EDM2014/uploads/procs2014/FengHeffernanKoedinger.pdf.

[7] J. S. Song, A. M. K. Choi, and J. Lee, *An Intelligent Tutoring System for Introductory C Language Programming*, Computers & Education, vol. 28, no. 2, pp. 93–108, 1997. Available: https://www.sciencedirect.com/science/article/pii/S0360131597000031.

[8] B. van de Sande, *Properties of the Bayesian Knowledge Tracing Model*, Journal of Educational Data Mining, vol. 5, no. 2, pp. 1–10, 2013. Available: https://jedm.educationaldatamining.org/index.php/JEDM/article/view/35.

[9] M. V. Yudelson, K. R. Koedinger, and G. J. Gordon, *Individualized Bayesian Knowledge Tracing Models*, Proceedings of the 16th International Conference on Artificial Intelligence in Education (AIED), pp. 171–180, 2013. Available: https://www.cs.cmu.edu/~ggordon/yudelson-koedinger-gordon-individualized-bayesian-knowledge-tracing.pdf.

[10] A. Robins, J. Rountree, and N. Rountree, *Learning and Teaching Programming: A Review and Discussion*, Computer Science Education, vol. 13, no. 2, pp. 137–172, 2003. Available: `https://www.tandfonline.com/doi/abs/10.1076/csed.13.2.137.14200`.

[11] K. R. Koedinger, A. T. Corbett, and C. Perfetti, *The Knowledge-Learning-Instruction Framework: Bridging the Science–Practice Chasm to Enhance Robust Student Learning*, Cognitive Science, vol. 36, no. 5, pp. 757–798, 2012. Available: `https://onlinelibrary.wiley.com/doi/10.1111/j.1551-6709.2012.01245.x`.